SIMATS ENGINEERING
ASSESSMENT TOOL 3: Mock Interview

COURSE NAME: COMPUTER VISION COURSE CODE: ITA0515 Slot B

COURSE OUTCOME COVERED
CO5: ***Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos.*** **(BTL 6)**

Assessment Tool Weightage: 10%

QUESTIONS:4 Total Marks 20 Duration :60 Minutes Annexure A
Mock Interview

Code of Conduct:

I _Ruthika Reddy C__ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. IL= understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Ruthika

Rubrics for Evaluation

Criteria	**Weightage (%)**	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**	***Marks(100)***
Technical Knowledge	25	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge	
Problem Solving	25	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problem solving ability; requires guidance	Unable to solve or apply concepts	
Analytical Thinking	15	Analyzes questions critically and provides well structured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated	
Communication	15	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively	

Confidence	15	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism	
Response to Questions	10	Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately	

Question 1

Design and explain a complete OpenCV pipeline to read, process, and display multiple images from a folder efficiently. Clearly justify your design choices and discuss potential challenges and optimizations.

Answer:
A complete OpenCV pipeline starts by identifying the image folder and obtaining all image file paths. I would use Python's os or glob module to collect files with extensions such as .jpg, .jpeg, and .png.

Then I would read each image using cv2.imread(). After reading, I would check whether the image was successfully loaded. The image can then be processed using operations such as resizing with cv2.resize(), converting color using cv2.cvtColor(), filtering, or edge detection.

After processing, I would display the image using cv2.imshow(). If there are multiple images, I would process them one by one instead of loading all large images into memory at once.

The basic pipeline is:

Folder → Get image paths → Read image → Validate → Process → Display → Release memory

For efficiency, I would resize large images before processing and process images sequentially. I would also avoid unnecessary conversions and use appropriate data types.

Potential challenges include invalid file paths, corrupted images, different image sizes, unsupported formats, and high memory usage. I would handle these using validation and exception checking.

Question 2

A video file is not loading properly in OpenCV. Analyze the possible causes and propose a structured debugging approach, explaining each step logically.

Answer:
First, I would verify that the video file path is correct. A wrong or missing path is one of the most common causes.

Next, I would create a VideoCapture object using cv2.VideoCapture() and check whether it opened successfully using cap.isOpened().

For example:

```
cap = cv2.VideoCapture("video.mp4")

if not cap.isOpened():
   print("Video could not be opened")
```

If the video opens, I would try reading frames using cap.read(). The return value tells me whether the frame was successfully retrieved.

I would then check the video format, codec, and file integrity. Some codecs may not be supported by the installed OpenCV/FFmpeg environment.

I would also check the video properties such as frame width, height, and FPS using cap.get().

My debugging sequence would be:

Check path → Check file existence → Check isOpened() → Check frame reading → Check codec/format → Check video properties → Test another video

Finally, I would release the resource using cap.release() and close windows using cv2.destroyAllWindows().

This structured approach helps identify whether the problem is related to the file, path, codec, OpenCV installation, or frame-processing code.

Question 3

You need to overlay text and shapes on a live video feed. Design your solution and explain how you would implement it efficiently, including key OpenCV functions and reasoning.

Answer:
I would capture the live video using cv2.VideoCapture(0). Inside a loop, I would continuously read frames from the camera.

Then I would overlay text and shapes directly on each frame. For text, I would use cv2.putText(). For shapes, I can use functions such as cv2.rectangle(), cv2.circle(), and cv2.line().

For example:

```
cv2.putText(frame, "Live Video",
        (20, 40),
        cv2.FONT_HERSHEY_SIMPLEX,
        1, (255, 255, 255), 2)

cv2.rectangle(frame, (50, 60), (250, 200),
         (255, 255, 255), 2)
```

After adding the overlays, I would display the frame using cv2.imshow().

I would use cv2.waitKey(1) to continuously update the display and provide a key such as q to exit.

The pipeline is:

Camera → Capture frame → Add text/shapes → Display frame → Repeat

For efficiency, I would avoid unnecessary image copying and expensive processing inside the loop. I would also process only the required region if possible.

At the end, I would call cap.release() and cv2.destroyAllWindows().

Question 4

An image appears too dark when read. Analyze the issue and propose modifications to your processing pipeline, justifying how your approach improves visibility.

Answer:
If an image appears too dark after reading, I would first check whether the image was loaded correctly using cv2.imread() and whether its pixel values are normal.

If the image is genuinely underexposed, I can improve its brightness and contrast.

One simple method is to increase brightness using image arithmetic. Another better approach is histogram-based enhancement.

For example, I can convert the image from BGR to grayscale using cv2.cvtColor() and then apply histogram equalization using cv2.equalizeHist().

For color images, I would prefer converting the image to a suitable color space such as LAB and applying contrast enhancement to the luminance channel. CLAHE, implemented using cv2.createCLAHE(), can improve local contrast while reducing excessive enhancement.

The pipeline would be:

Read image → Check image → Analyze brightness → Enhance brightness/contrast → Display result

I would choose the enhancement method based on the problem. Simple brightness adjustment is suitable for a mildly dark image, while histogram equalization or CLAHE is better when contrast is also poor.

This improves visibility while attempting to preserve important image details.